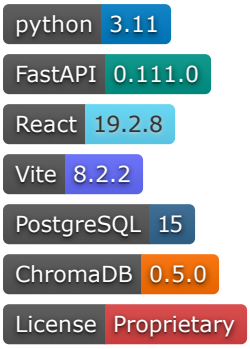
BusinessIntelligence.ai

Evidence-Backed KPI Decision Engine & Causal Governance Architecture



---

1. Executive Overview

The Problem

Modern enterprise observability and Business Intelligence dashboards suffer from a fundamental epistemic disconnect:
1. **Dashboards Show *What* Changed, Not *Why*:** Traditional BI tools display metric drops (e.g. conversion down 14%, latency up 300%) but cannot isolate whether the root cause was an internal deployment, an upstream payment gateway outage, a competitor marketing campaign, or an ETL ingestion telemetry artifact.
2. **Generic LLMs Hallucinate Causal Chains:** Standard AI assistants generate convincing qualitative stories but lack deterministic mathematical grounding. When presented with correlation, they jump to conclusions, propose ungrounded production interventions (e.g. rolling back software during a third-party ISP outage), and ignore enterprise governance boundaries.
3. **Absence of Epistemic Guards & Decision Rights:** Existing systems lack mathematical abstention mechanisms. When data is sparse (<14 days), ambiguous, or corrupted, generic AI guesses rather than abstaining.

The Solution

BusinessIntelligence.ai is an evidence-backed KPI decision engine designed for enterprise reliability engineering, financial operations, and executive decision-making.

The platform separates **deterministic mathematical auditing and content-hashed evidence retrieval** from **LLM qualitative hypothesis synthesis and executive briefing**. Every candidate hypothesis is put on trial against 5 deterministic non-LLM verification rules and hard epistemic gates. The system enforces strict role-based data entitlements (`Analyst`, `CFO`, `Manager`), adheres to governed operational levers, and triggers automated safety abstentions whenever evidence confidence is marginal or ambiguous.

---

2. Solution Architecture: The 9-Engine Investigation Pipeline

The platform processes incidents through a strictly governed 9-stage pipeline ($E_1 \rightarrow E_9$):

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                  9-ENGINE INVESTIGATION PIPELINE                             │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

  [E1: KPI Store]          → Deterministic SQL query & rolling baseline corridors (|z| < 3.0σ)
        ↓
  [E2: Signal Engine]      → Anomaly detection & revenue materiality prioritization (W_rev × Δ)
        ↓
  [E3: Diagnostic Engine]  → Multi-dimensional SQL decomposition (device, region, channel cohorts)
        ↓
  [E4: Evidence Dossier]   → Hybrid vector (ChromaDB) + SQL relational retrieval with role entitlements
        ↓
  [E5: Hypothesis Studio]  → LLM synthesis of candidate explanations under a 4-layer qualitative ontology
        ↓
  [E6: Challenge Engine]   → 100% Deterministic constraint auditing (5 rules + Root-Cause Evidence Gate)
        ↓
  [E7: Decision Engine]    → Governed action formulation (Authorized Levers vs. Safety Abstention Guards)
        ↓
  [E8: Outcome Simulator]  → Calibrated exponential recovery curves & Mean Time to Normalcy [SIMULATED]
        ↓
  [E9: Memory Engine]      → Vector institutional memory (HNSW/Cosine) & 5-state precedent lifecycle
```

Why This Architecture?

The architecture is built on a core principle: **never ask an LLM to perform mathematical auditing, and never force a database to formulate qualitative domain context.**

```
Quantitative truth       → Relational SQL / Rolling Statistics / Corridor Math
Qualitative explanation  → Large Language Models (LLM) under strict schemas
Causal verification      → Deterministic 5-rule audit & weakest-link scoring
Operational decision     → Policy rules + Role entitlements + LLM briefing
Expected outcome         → Calibrated parametric simulation
Institutional learning   → Vector retrieval + Human feedback lifecycle
```

Detailed Engine Breakdown

Engine	Name	Primary Responsibility	Input	Output	Processing Method	Governance & Safety Safeguards
E1	**KPI Store**	Ingests time-series telemetry and computes historical rolling baseline corridors (μ, σ).	KPI Contract YAML, Time Window $[t_0, t_1]$, Scope filter	Normalized Time-Series DataFrames	**Deterministic SQL / Pandas**	Statistical Cold-Start Guard (flags datasets with < 14 intervals of baseline history).
E2	**Signal Engine**	Detects statistical anomalies (z-score) and computes revenue materiality rankings.	E1 Time-Series Data	Prioritized `list[AnomalySignal]` with materiality tiers	**Deterministic Statistical Math**	Adverse direction filtering; data quality anomaly detection (ETL ingestion delay vs real drop).
E3	**Diagnostic Engine**	Decomposes material anomalies across dimensional slices (`device`, `region`, `channel`).	E2 Top Prioritized Anomaly, SQL Transaction Store	`list[DimensionContribution]` (cohort contribution %)	**Deterministic SQL GROUP BY Aggregation**	Multi-dimensional variance isolation; zero-segment safe handling without crashing.
E4	**Evidence Dossier**	Assembles verifiable telemetry records, deployment logs, and vendor feeds.	E2/E3 Signals, Persona Entitlements YAML	`list[Evidence]` with content hashes and timestamps	**Hybrid Vector (ChromaDB) + Relational Entitlement Filter**	Strict role-based entitlement filtering (masks SRE/deployment logs from non-technical personas).
E5	**Hypothesis Studio**	Formulates candidate causal explanations structured under a 4-layer ontology.	E2 Signals, E3 Contributions, E4 Evidence Dossier	`list[Hypothesis]` with mechanisms & evidence citations	**LLM Synthesis (Groq Qwen / Ollama Llama)**	Zero-number qualitative propositions; prompt-enforced prohibition of ungrounded citation hashes.
E6	**Challenge Engine**	Audits candidate hypotheses against 5 deterministic rules and epistemic gates.	E5 Hypotheses, E4 Evidence Records, Anomaly Windows	`list[ScoredHypothesis]` with `AuditVerdict` & rule scores	**100% Deterministic Pure Mathematical Functions**	**Root-Cause Evidence Gate:** Mandatory discriminative evidence for release/provider claims; Weakest-link constraint.
E7	**Decision Engine**	Synthesizes governed operational action directives or	E6 Audited Scorecards, Entitlements, Controllable Levers	`DecisionPayload` with action directive & persona narrative	**Deterministic Policy Rules + LLM Executive Briefing**	**Fail-Closed Abstention:** Automated mitigation suppressed if score

Engine	Name	Primary Responsibility	Input	Output	Processing Method	Governance & Safety Safeguards
		triggers safety guards.				< 0.70, margin < 0.15, or lever unauthorized.
E8	**Outcome Simulator**	Projects recovery trajectories and Mean Time to Normalcy (MTTN) under proposed actions.	E7 Action Directive, E2 Anomaly Baseline Delta	`OutcomeProjection` with recovery curves $[y(t)]$	**Deterministic Calibrated Exponential Decay Simulation**	Explicit `[SIMULATED]` provenance tag; non-remedial actions suppress recovery projections; causal disclaimer.
E9	**Memory Engine**	Embeds verified incident cases and retrieves lifecycle-filtered institutional precedents.	E7 Decision, Analyst Feedback Record, ChromaDB	Historical precedent citations & matching score	**Vector Embeddings (ChromaDB / Cosine Similarity)**	**5-State Precedent Lifecycle:** Validated precedents receive ranking boosts; disputed/suppressed records are excluded.

3. LLM vs. Non-LLM Processing Matrix

To guarantee mathematical reproducibility and enterprise auditability, the system strictly isolates stochastic LLM generation from deterministic computation:

Pipeline Stage	Method	LLM Required?	Exact Role of LLM vs. Deterministic Code
E1 KPI Store	Relational SQL & Rolling Math	❌ **No**	100% deterministic database queries and statistical baseline computation (μ, σ).
E2 Signal Detection	Statistical z-Score & Materiality	❌ **No**	100% deterministic formula: $z = (x_t - \mu)/\sigma$; Materiality = $W_{rev} \times \Delta$.
E3 Diagnostic Decomposition	Relational Dimensional Slicing	❌ **No**	100% deterministic SQL group-by aggregation across device, region, and channel cohorts.
E4 Evidence Dossier	Hybrid Vector + Relational Security	⚠️ **Embeddings Only**	Dense embedding inference for semantic search; 100% deterministic entitlement masking.
E5 Hypothesis Studio	Causal Ontology Formulation	✅ **Yes**	LLM generates qualitative explanations conforming to the 4-layer causal schema.
E6 Challenge Audit	Deterministic Constraint Scoring	❌ **No**	**Zero LLM involvement in audit scoring.** Evaluated via 5 non-LLM Python rules and hard gates.
E7 Decision Engine	Governed Action Formulation	⚠️ **Hybrid**	Deterministic rule decides `VERIFIED` vs `ABSTAIN`; LLM synthesizes role-adapted narrative.
E8 Outcome Simulation	Exponential Recovery Decay	❌ **No**	100% deterministic parametric simulation: $y(t) = y_{target} + (y_0 - y_{target})e^{-\lambda t}$.
E9 Memory Engine	Precedent Vector Store	⚠️ **Embeddings Only**	Dense embedding generation; deterministic cosine/HNSW matching and 5-state lifecycle filtering.

Early Deterministic Guards (LLM Bypass)

To optimize LLM economics and eliminate hallucinations, deterministic early guards can bypass LLM inference entirely for scenarios that fail data-quality, baseline, or nominal-corridor checks:
* **Cold-Start Guard (**< 14 **Days):** If baseline history is insufficient, hypothesis generation is skipped (0 LLM calls).
* **Nominal Corridor Guard ($|z| < 3.0\sigma$)**: ** $If all telemetry streams fluctuate normally within calibrated corridor bounds, the system declares 'SYSTEM NOMINAL'$ **(0 LLM calls).**
* Data Quality Guard (ETL Delay):** If revenue drops without application error or gateway latency elevation, the system triggers the Data Quality Guard without invoking speculative LLM remediations.

---

4. Causal Reasoning & Epistemic Audit Design

4-Layer Causal Ontology

The system enforces a strict 4-layer qualitative ontology to prevent conflating symptoms with causes:

```
┌──────────────────────────────────────────────────────────────────────┐
│                          4-LAYER CAUSAL ONTOLOGY                     │
├──────────────────────────┬───────────────────────────────────────────┤
│ 1. ROOT CAUSE            │ The initiating event (e.g. INTERNAL_RELEASE,  │
│                          │ EXTERNAL_PROVIDER, INFRASTRUCTURE_FAILURE)    │
├──────────────────────────┼───────────────────────────────────────────┤
│ 2. AFFECTED SUBSYSTEM    │ The architectural component experiencing stress │
│                          │ (e.g. payment_gateway, checkout_service, inventory_db) │
├──────────────────────────┼───────────────────────────────────────────┤
│ 3. PROXIMAL MECHANISM    │ The physical/technical failure mode           │
│                          │ (e.g. connection_pool_exhaustion, memory_leak, packet_loss)│
├──────────────────────────┼───────────────────────────────────────────┤
│ 4. SYMPTOM KPIs          │ The observable statistical telemetry anomalies │
│                          │ (e.g. gateway_latency_15min, hourly_conversion, revenue) │
└──────────────────────────┴───────────────────────────────────────────┘
```

Example Verified Chain:
`INTERNAL_RELEASE` (Checkout v4.3) → `payment_gateway` → `connection_pool_exhaustion` → `gateway_latency_15min (+3.44σ) & conversion drop (-14.2%)`.

Stage E6: Deterministic Audit Architecture & Weakest-Link Scoring

Audit scoring is a deterministic function of evidence, rule verdicts, and configured weights. In Stage E6, every candidate hypothesis is audited against **5 deterministic verification rules**:

1. **`timeline`** ($w_1 = 0.25$)**:** Validates that initiating event timestamps precede the anomaly window ($t_{evidence} \leq t_{anomaly}$).
2. **`segment_alignment`** ($w_2 = 0.20$)**:** Validates that the hypothesis aligns with dimensional cohort concentrations identified in E3.
3. **`kpi_corroboration`** ($w_3 = 0.20$)**:** Measures the proportion of anomalous KPIs directly explained by the cited evidence records.
4. **`mechanism_consistency`** ($w_4 = 0.20$)**:** Validates ontological compatibility between the proposed mechanism and known domain failure modes.
5. **`contradiction`** ($w_5 = 0.15$)**:** Evaluates the presence of direct refuting evidence records in the dossier.

The Weakest-Link Scoring Formula

The engine computes the final score using a weakest-link formula to prevent strong evidence from masking rule failures:

$$\text{rule\_score} = \sum_{i=1}^{5} w_i \times \text{Multiplier}(\text{Verdict}_i) \quad \text{where Multiplier} \in \{1.0\ (\text{PASS}), 0.5\ (\text{PARTIAL}), 0.0\ (\text{FAIL})\}$$

$$\text{capped\_support} = \text{clamp}\left(\frac{\text{support\_score}}{2.0}, 0.0, 1.0\right)$$

$$\text{capped\_penalty} = \text{clamp}\left(\frac{\text{contradiction\_score}}{2.0}, 0.0, 1.0\right)$$

$$\text{final\_audit\_score} = \text{clamp}\left(\min(\text{capped\_support}, \text{rule\_score}) - \text{capped\_penalty}, 0.0, 1.0\right)$$

The Root-Cause Evidence Gate

Under enterprise audit rules, claiming an `INTERNAL_RELEASE` or `EXTERNAL_PROVIDER` root cause requires **direct discriminative evidence** (e.g. git commit logs, CI/CD deployment records, third-party status page logs).
* **If Discriminative Evidence is Missing:** The Root-Cause Gate **FAILS**, and the hypothesis audit verdict is automatically **CAPPED AT `MARGINAL` (0.40)**, preventing the system from blaming external vendors or internal software without empirical proof.

AuditVerdict Semantics

- **`VERIFIED`** (≥ 0.70)**:** Deterministically verified causal chain with sufficient evidence and passed Root-Cause Gate.

- `MARGINAL` **($0.40 - 0.69$):** Partially corroborated hypothesis. Autonomous remediation is blocked; safe diagnostic verification protocols are formulated.
- `REJECTED` **(< 0.40):** Refuted by contradictory evidence or failed verification rules.
- `ABSTAIN`: Investigation-level verdict triggered when the top hypothesis is below threshold (< 0.40), the margin between competing hypotheses is too narrow ($\Delta < 0.15$), or data quality/cold-start guards activate.

---

5. Security, Governance & Role Entitlements

The platform enforces zero-trust data governance across three enterprise persona scopes:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      ROLE ENTITLEMENT & VISIBILITY MATRIX                │
├──────────────────┬──────────────────────────────┬────────────────────────┤
│ Persona Scope    │ Authorized Data Sources      │ Authorized Operational Levers │
├──────────────────┼──────────────────────────────┼────────────────────────┤
│ Lead Analyst     │ ALL (deployments, logs, SRE, │ Software Rollback, Traffic Reroute│
│ (Full System)    │ orders, inventory, marketing) │ Canary Diagnostic Verification │
├──────────────────┼──────────────────────────────┼────────────────────────┤
│ CFO / Executive  │ orders, marketing, payment   │ Budget Realignment, Promotional │
│ (Financial Scope)│ gateway aggregates (No SRE)  │ Hold, Executive Escalation │
├──────────────────┼──────────────────────────────┼────────────────────────┤
│ Regional Manager │ inventory, orders for scoped │ Inventory Rebalance, Regional │
│ (Restricted Scope)│ geographic region (No SRE)  │ Promotion, Diagnostic Escalation │
└──────────────────┴──────────────────────────────┴────────────────────────┘
```

Security Controls Implemented

- **Fail-Closed Governance:** If a persona attempts to formulate an action using an unallowed lever (or outside their authorization scope), execution is blocked pending architecture review.
- **Anti-Hallucination Citation Validation:** All cited evidence IDs are validated against the content-hashed, provenance-tracked database records assembled in E4. Ghost citations are rejected with a 100% score penalty.
- **Regional Data Boundary Enforcement:** Slices queries strictly to authorized regions (`us-east`, `us-west`, `eu-west`, `ap-south`, `all`), preventing cross-regional telemetry leakage.

---

6. Outcome Simulation (Stage E8)

Stage E8 generates parametric recovery projections under approved operational interventions:

- **Provenance Label:** Explicitly tagged as `[SIMULATED]` across all API responses and UI components to prevent confusing simulated projections with observed real-world telemetry.
- **Parametric Recovery Model:** Modeled using calibrated exponential decay towards baseline target corridors:

$$y(t) = y_{\text{target}} + (y_{\text{anomaly}} - y_{\text{target}}) \cdot e^{-\lambda t}$$

- **Mean Time to Normalcy (MTTN):** Calculated as the duration required for the metric to recover within $\pm 1.0\sigma$ of historical baseline.
- **Non-Remedial Suppression:** When the engine abstains or issues a non-remedial diagnostic action, recovery curves are suppressed, and an explicit causal disclaimer is attached.

---

7. Institutional Memory & Precedent Lifecycle (Stage E9)

Stage E9 manages long-term institutional knowledge in a ChromaDB vector store:

- **Lifecycle-Filtered Vector Retrieval:** Precedent queries retrieve historical records, with human-validated records receiving a ranking boost, unvalidated records included under standard similarity weighting, and disputed/suppressed records **strictly excluded from candidate sets**.
- **5-State Precedent Lifecycle:**
  1. `UNVALIDATED`: Newly completed investigation pending peer review (eligible for retrieval under baseline weighting).
  2. `VALIDATED`: Approved by senior analyst / SRE (eligible for retrieval with ranking boost).
  3. `PARTIALLY_VALIDATED`: Confirmed mechanism, modified action directive.
  4. `DISPUTED`: Rejected during human review (**strictly excluded from retrieval**).
  5. `SUPPRESSED`: Deprecated due to architectural redesign or outdated infrastructure (**strictly excluded from retrieval**).
- **Closed-Loop Feedback:** Operators submit structured reviews (`POST /feedback`) to update precedent lifecycle states and prevent error propagation.

---

8. Data Ingestion Architecture & ChromaDB Population

1. Relational Telemetry Store (PostgreSQL)

- Ingests structured time-series KPI intervals, orders, inventory logs, and payment transactions defined in `etl/schema.sql`.
- Populated via `etl/load_synthetic.py` (with fallback to DuckDB/in-memory SQLite for localized standalone testing).

2. Vector Evidence & Precedent Store (ChromaDB)

The vector store manages two distinct categories of collections in ChromaDB (Port 8000):

- **Scenario Evidence Collections (`evidence_{scenario_id}`):**
- Stores unstructured telemetry evidence: customer support tickets (`support_tickets.csv`), deployment logs (`deployment_log.csv`), and engineering release notes (`data/release_notes/*.txt`).
- Embedded using `bge-m3` via Ollama (`http://localhost:11434`) or ChromaDB's default dense embedding function.

- **Population Commands:**
  ```bash
  # Embed and load core unstructured evidence records (INC_001)
  python etl/load_unstructured.py --scenario-id INC_001 --chroma-host localhost --chroma-port 8000

# Seed multi-scenario evidence collections (INC_002, INC_004, etc.)

  python etl/seed_scenario_evidence.py --chroma-host localhost --chroma-port 8000
  ```

- **Institutional Memory Collections (`bi_decisions_precedents`):**

- Stores completed incident investigation cases, verified winning hypotheses, and structured recommendations.
- Automatically populated at runtime by **Engine E9 (`engines/memory.py`)** after each successful investigation run.
- Maintains the 5-state validation lifecycle (`UNVALIDATED`, `VALIDATED`, `PARTIALLY_VALIDATED`, `DISPUTED`, `SUPPRESSED`).

3. Live Streaming Ingestion Boundary

- The current system operates against historical interval batches and staged incident datasets. High-frequency Kafka / event-streaming ingestion pipelines are not implemented in this prototype.

---

9. Scenarios Catalog

Core Validated Reference Scenarios

Scenario ID	Incident Name	Business Situation	Expected Engine Behavior	Final Decision State
`INC_001`	**Payment Gateway Latency Regression**	Checkout Service v4.3 release exhausted connection pools, inducing latency spikes (+3.44σ) and conversion drops.	Full deterministic causal chain verified against git release logs and gateway error metrics.	**Overall Verdict:** `VERIFIED` Winner: `H1` (Score: 71%) Action: Immediate software rollback to v4.2 authorized.
`INC_002`	**Simultaneous Conflicting Causes**	Concurrent payment gateway timeouts and aggressive competitor 30% discount.	Narrow margin between competing hypotheses triggers Multi-Causal Conflict Guard.	**Overall Verdict:** `ABSTAIN` Winner: `None` (`H1` = Marginal 80%, `H2` = Marginal 80%) Action: Governed diagnostic verification / no operational remediation.
`INC_003`	**Sparse Baseline History**	Monitored growth domain has fewer than 14 days of baseline telemetry.	Statistical Cold-Start Guard triggers early bypass (0 LLM calls) to prevent false-positive anomaly spikes.	**Overall Verdict:** `ABSTAIN` Winner: `None` Reason: Statistical Cold-Start Guard (<14 intervals).
`INC_004`	**ETL Ingestion Pipeline Delay**	Upstream Kafka/Airflow ETL lag causes missing interval batches, mimicking a revenue collapse.	Data Quality Guard detects 0% gateway error rates and flags missing batch artifacts vs real sales loss.	**Overall Verdict:** `ABSTAIN` Winner: `None` Reason: Data Quality Guard (ETL Ingestion Lag).
`INC_005`	**Seasonal Demand Pattern**	Standard seasonal traffic fluctuations within historical corridor bounds ($	z	< 3.0\sigma$).

Additional Demonstration Scenarios in Catalog

Scenario ID	Incident Name	Demonstration Focus	Engine Behavior
`INC_006`	**Compound Network & Deploy Failure**	Dual-event infrastructure stress.	Root-Cause Gate caps external claims at marginal; Governance Guard authorizes canary diagnostic verification.
`INC_007`	**Gradual Worker Memory Leak**	Low-frequency background worker degradation.	Deterministic rules audit candidate explanation; safely abstains under low confidence (27%).
`INC_008`	**Enterprise SAML SSO Outage**	Third-party identity provider failure.	Multi-source authentication logs correlated; role entitlement scope enforced.

10. Dependencies & Technology Stack

Backend

- **Python Version:** `Python 3.11.9`
- **Core Framework:** `fastapi==0.111.0`, `uvicorn==0.29.0`
- **Data Processing & Math:** `pandas==2.2.2`, `numpy==1.26.4`, `scipy==1.13.0`, `sqlalchemy==2.0.30`
- **Vector Database:** `chromadb==0.5.0`
- **Relational Database:** `psycopg2-binary==2.9.9` (PostgreSQL 15+)
- **Configuration & HTTP:** `pyyaml==6.0.1`, `python-dotenv==1.0.1`, `httpx==0.27.0`
- **Testing:** `pytest==8.2.1`, `pytest-asyncio==0.23.7`, `hypothesis==6.103.1`

Frontend

- **Node.js & Runtime:** `Node.js ⩾ 18.0.0`, `npm ⩾ 9.0.0`
- **Core UI Framework:** `react@19.2.8`, `react-dom@19.2.8`, `typescript@6.0.2`, `vite@8.2.0`
- **Styling & Design System:** `tailwindcss@3.4.19`, `clsx@2.1.1`, `tailwind-merge@3.6.0`
- **UI Primitives & Motion:** `@radix-ui/react-dialog`, `@radix-ui/react-tabs`, `framer-motion@13.1.1`, `motion@13.1.1`, `animejs@4.5.0`, `lucide-react@1.33.0`
- **Data Visualization:** `recharts@3.10.1`
- **Frontend Testing:** `vitest@4.1.11`, `@testing-library/react@16.3.2`, `@testing-library/jest-dom@7.0.1`, `jsdom@29.1.1`

Infrastructure

- **PostgreSQL (Port 5432):** Mandatory for primary relational telemetry storage.
- **ChromaDB (Port 8000):** Vector database for unstructured evidence dossier and precedent memory.
- **LLM Providers:**
- *Cloud / High-Speed:* Groq API (`qwen/qwen3.8-27b` or `llama-3.3-70b-versatile`).
- *Local Offline Alternative:* Ollama (`llama3.2` or `qwen2.5:7b` on `http://localhost:11434`).

---

11. Configuration & Environment Variables

Copy `.env.example` to `.env` in the root directory:

```
# ============================================================================
# DATABASE & STORAGE CONFIGURATION
# ============================================================================
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/bi_decisions
CHROMA_HOST=localhost
CHROMA_PORT=8000

# ============================================================================
# LLM INFERENCE PROVIDER
# Options: 'groq' | 'ollama'
# ============================================================================
LLM_BACKEND=groq

# Groq Cloud Configuration
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=qwen/qwen3.8-27b

# Ollama Local Configuration (Fallback)
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2

# ============================================================================
# API SERVER CONFIGURATION
# ============================================================================
API_PORT=8085
API_HOST=0.0.0.0
```

---

12. Execution Guide

1. Start Infrastructure Services

```
# Start PostgreSQL and ChromaDB via Docker Compose
docker-compose up -d
```

2. Initialize Database & Seed Scenario Telemetry & ChromaDB

```
# 1. Load relational schema and synthetic telemetry into PostgreSQL
python etl/load_synthetic.py

# 2. Embed and populate unstructured evidence into ChromaDB
python etl/load_unstructured.py --scenario-id INC_001
python etl/seed_scenario_evidence.py
```

3. Start Backend API Server

```
# Launch FastAPI on port 8085 with live hot-reloading
uvicorn api.main:app --host 0.0.0.0 --port 8085 --reload
```

Interactive Swagger Documentation available at: `http://localhost:8085/docs`

4. Start React Operations Console

```
# Navigate to web frontend directory
cd web

# Install dependencies and start Vite dev server
npm install
npm run dev
```

Web Application available at: `http://localhost:5173`

---

13. How to Execute an Investigation

Via Web UI

1. Open `http://localhost:5173`.
2. Select an incident scenario from the top dropdown (e.g. `INC_001 Payment Gateway Latency`).
3. Select an Analyst Persona (`Analyst`, `CFO`, `Manager`) and Region Scope (`Global`, `US-East`, etc.).
4. Click **Run Investigation** to watch the 9-stage pipeline execute with live streaming stage cards.
5. Inspect the E6 Rule Scorecard, Root-Cause Evidence Gate, and E7 Governed Action Directive.

Via API Endpoint (cURL)

```
curl -X POST "http://localhost:8085/investigate" \
  -H "Content-Type: application/json" \
  -d '{
    "scenario_id": "INC_001",
    "persona": "analyst",
    "region": "all"
  }'
```

---

14. Testing & Verification Strategy

1. Automated Backend Pytest Suite

```
# Run unit, integration, and security verification tests
pytest -v --tb=short
```

2. Frontend Vitest Component Suite

```
# Run React Testing Library & Vitest test suites (12 suites, 35/35 passing)
cd web
npm test
```

3. Frontend Production Build Verification

```
# Compile TypeScript bundle and verify zero build errors
cd web
npm run build
```

4. Target Matrix Verification Suite

```
# Execute 26-run multi-scenario and multi-persona matrix suite with rate-limit protection
python scratch/run_full_matrix_verification.py
```

---

15. Runtime Telemetry & LLM Economics

The platform provides granular telemetry tracking across every pipeline run:

- **Latency Tracking:** Measures millisecond execution time across individual stages (E_1 through E_9).
- **Token Accounting:** Tracks precise prompt tokens (`llm_tokens_in`), completion tokens (`llm_tokens_out`), and total token consumption.
- **Cost Estimator (`llm/cost_estimator.py`):** Dynamically computes dollar cost per investigation based on active model rate cards (e.g. Groq Qwen USD 0.05 / 1M tokens vs. OpenAI GPT-4o).
- **Deterministic Efficiency:** By leveraging deterministic early guards (bypassing LLMs for cold-start and nominal states), the engine eliminates unnecessary LLM inference costs for un-actionable telemetry runs.

---

16. Known Limitations

- **Batch vs Streaming Ingestion:** Telemetry is analyzed across fixed historical windows (15m to 24h); sub-second streaming event ingestion is not implemented in this prototype.
- **Simulated Recovery Projections:** Stage E8 recovery trajectories are mathematical exponential approximations and should not be interpreted as guaranteed real-world SLA outcomes.
- **Vector Semantic Scope:** Unstructured evidence retrieval depends on the quality and density of ingested log summaries and incident postmortems.

---

17. Repository Structure

```
BusinessIntelligence.ai/
├── api/
│   ├── __init__.py
│   └── main.py                      # FastAPI application endpoints (/investigate, /feedback, /health)
├── config/
│   ├── business_denylist.yaml       # Prohibited speculative actions & unallowed levers
│   ├── domain_semantics.yaml        # Domain ontology mappings & mechanism taxonomy
│   ├── entitlements.yaml            # Role-based persona access policies & source scopes
│   ├── evidence_mappings.yaml       # Source-to-mechanism correlation weights
│   ├── kpi_contracts.yaml           # KPI definitions, directions, and baseline corridors
│   ├── loader.py                    # Strict YAML schema validation & configuration loader
│   ├── memory_retention.yaml        # Vector retention thresholds & invalidation rules
│   ├── registry.py                  # Source and entity registry loader
│   └── scenarios.yaml               # Demonstration incident scenario catalog (INC_001 - INC_008)
├── engines/
│   ├── challenge.py                 # Engine E6: Deterministic Constraint Auditing & Root-Cause Gate
│   ├── decision.py                  # Engine E7: Governed Decision Formulation & Action Directives
│   ├── diagnostic.py                # Engine E3: Multi-Dimensional Slicing & Decomposition
│   ├── evidence.py                  # Engine E4: Grounded Evidence Dossier Hybrid Retrieval
│   ├── hypothesis.py                # Engine E5: 4-Layer Causal Ontology Hypothesis Formulation
│   ├── kpi_store.py                 # Engine E1: Time-Series KPI Telemetry Ingestion & Baselines
│   ├── memory.py                    # Engine E9: Vector Institutional Memory & 5-State Lifecycle
│   ├── outcome.py                   # Engine E8: Parametric Exponential Outcome Simulation
│   └── signal.py                    # Engine E2: Anomaly Detection & Revenue Materiality Ranking
├── etl/
│   ├── generate_scenarios.py        # Synthetic incident generator & time-series seed data
│   ├── load_synthetic.py            # Relational telemetry database loader
│   ├── load_unstructured.py         # Vector database embedding loader (ChromaDB)
│   └── schema.sql                   # PostgreSQL relational schema definition
├── llm/
│   ├── cost_estimator.py            # Per-run token accounting & LLM inference cost estimator
│   ├── provider.py                  # Groq & Ollama LLM provider client abstractions
│   └── telemetry_wrapper.py         # Latency & token monitoring wrapper
├── pipeline/
│   ├── investigate.py               # 9-engine execution orchestrator & dependency injector
│   └── telemetry.py                 # Pipeline telemetry aggregation models
├── security/
│   └── entitlements.py              # Server-side role entitlement & source masking engine
├── tests/
│   ├── test_causal_e6_isolation.py # E6 deterministic scoring isolation test suite
│   ├── test_challenge_smoke.py      # E6 5-rule constraint verification tests
│   ├── test_feedback.py             # E9 analyst feedback & lifecycle state transition tests
│   ├── test_security.py             # Role-based entitlement & access control tests
│   └── test_signal.py               # E1/E2 statistical corridor & anomaly detection tests
├── web/
│   ├── src/
│   │   ├── components/
│   │   │   ├── decision/            # E7 Decision Hero & Abstention Card views
│   │   │   ├── engines/             # Workspaces for Stages E1 through E9
│   │   │   ├── hypothesis/          # E5/E6 Hypothesis Cards & Rule Scorecard components
│   │   │   ├── investigation/       # Pipeline Rail, Feedback Bar, Scenario Selector
│   │   │   ├── kpi/                 # KPI Signal Grid & telemetry cards
│   │   │   └── layout/              # TopBar, LeftObservePanel, RightBeliefPanel
│   │   ├── lib/
│   │   │   ├── api.ts               # Backend API client
│   │   │   ├── narrativeHelpers.ts # 100% dynamic payload narrative presenter
│   │   │   └── utils.ts             # Metric formatters & LLM tag cleaners
│   │   ├── types/
│   │   │   └── investigation.ts     # Complete TypeScript schema definitions
│   │   ├── App.tsx                  # Main application view container
│   │   └── index.css                # Design system tokens & dark theme styling
│   ├── package.json                 # Frontend package dependencies & scripts
```

```
│   └── vite.config.ts              # Vite configuration & proxy definitions
├── docker-compose.yml              # PostgreSQL and ChromaDB container definitions
├── models.py                       # Core Python dataclass models and type definitions
├── requirements.txt                # Python backend package dependencies
└── README.md                       # Complete technical framework documentation
```

---

18. Submission & Technical Review

This repository represents the completed Round 2 implementation of **BusinessIntelligence.ai**. It demonstrates:
* **Mathematical Causal Grounding:** Confidence scores derived exclusively from deterministic rules rather than stochastic LLM tokens.
* **Governed Enterprise Safety:** Automatic abstentions during cold-start, multi-causal ambiguity, and data quality lag.
* **Role-Based Security:** Server-side entitlement enforcement across Analyst, CFO, and Regional Manager roles.
* **Enterprise-Oriented Analytical Console:** Dark-mode console with sub-second responsive interaction states and verified test suites.

Submitted for evaluation — BusinessIntelligence.ai Technical Architecture Team.